

XIAMEN UNIVERSITY

COM102 – Programming Language C

Lecture 1 – Week 1

About the Course

Dr. Najla'a

Outline

- Instructor
- Grading Criteria
- Course Information
- Recommended Textbook

Course Information

Dr. Najla`a Ateeq Mohammed Draib

School of Computing and Data Science, Xiamen University
najlaaateeqmohammed.draib@xmu.edu.my

- **PhD** in Cyber Security, University Putra Malaysia
- **MSc** in Software engineering, University Putra Malaysia
- **BSc** in Computer Science , Thamar University, Yemen
- **Postgraduate Diploma** in Education, Al-Madinah international university, Malaysia

Consultation Time

Office: A1-351

Consultation hours:

- ✓**Monday** **10 AM – 12 PM,**
- ✓**Tuesday** **10 AM – 12 PM,**

or make an appointment

Grading Criteria

Assessment Methods and Types:

Method of Assessment	Total
Continuous Assessment	
· Quiz	30%
· Assignment	20%
Final Assessment	
· Final Exam	50%
· Total	100%

Assessment

Method of Assessment	Estimated Time
Continuous Assessment	
· Quiz	Week 7
· Assignment	Week 8
Final Assessment	
• Final Exam	Set by the Exam Unit

XIAMEN UNIVERSITY

Grading System

- Grading Scales and grades applicable to ALL intakes effective from Academic Session 2017/09:

Grading System for Academic Session 2017/09 onwards			
Marks	**Grade**	**Grade Points**	**Description**
85 – 100	A	4.00	Distinction
80 – 84	A-	3.70	Distinction
75 – 79	B+	3.30	Good
70 – 74	B	3.00	Good
65 – 69	B-	2.70	Good
60 – 64	C+	2.30	Pass
55 – 59	C	2.00	Pass
50 - 54	C-	1.70	Conditional Pass
45 - 49	D+	1.30	Fail
40 - 44	D	1.00	Fail
≤ 39	F	0.00	Fail

Table 1: Grading System Effective from Academic Session 201709

What is this class about?

This course is an introduction to programming

It provides a thorough introduction to the C programming language.

It covers basic syntax and grammar, and exposes students to practical programming techniques

XIAMEN UNIVERSITY

Course Learning Outcomes

Describe the C-language syntax.

Analyze and debug programming code.

Design codes to solve practical problems using C language.

Credit Value

- 3

Course Objectives

- To provide foundation of the computer programming in using conditional and iterative structures, functional decomposition, and basic parallelization techniques.
- To provide students with basic knowledge of programming in C.
- To help the students to learn to write some commonly used algorithms and subroutines, such as find the maximum from an array, bubble sorting, string manipulating, pointer operation, etc.
- To prepare students for their future work in the development of large scale software systems.



Course Outline

01 Introduction

02 Types, Operators and Expressions

03 Control Flow

04 Functions and Program Structure

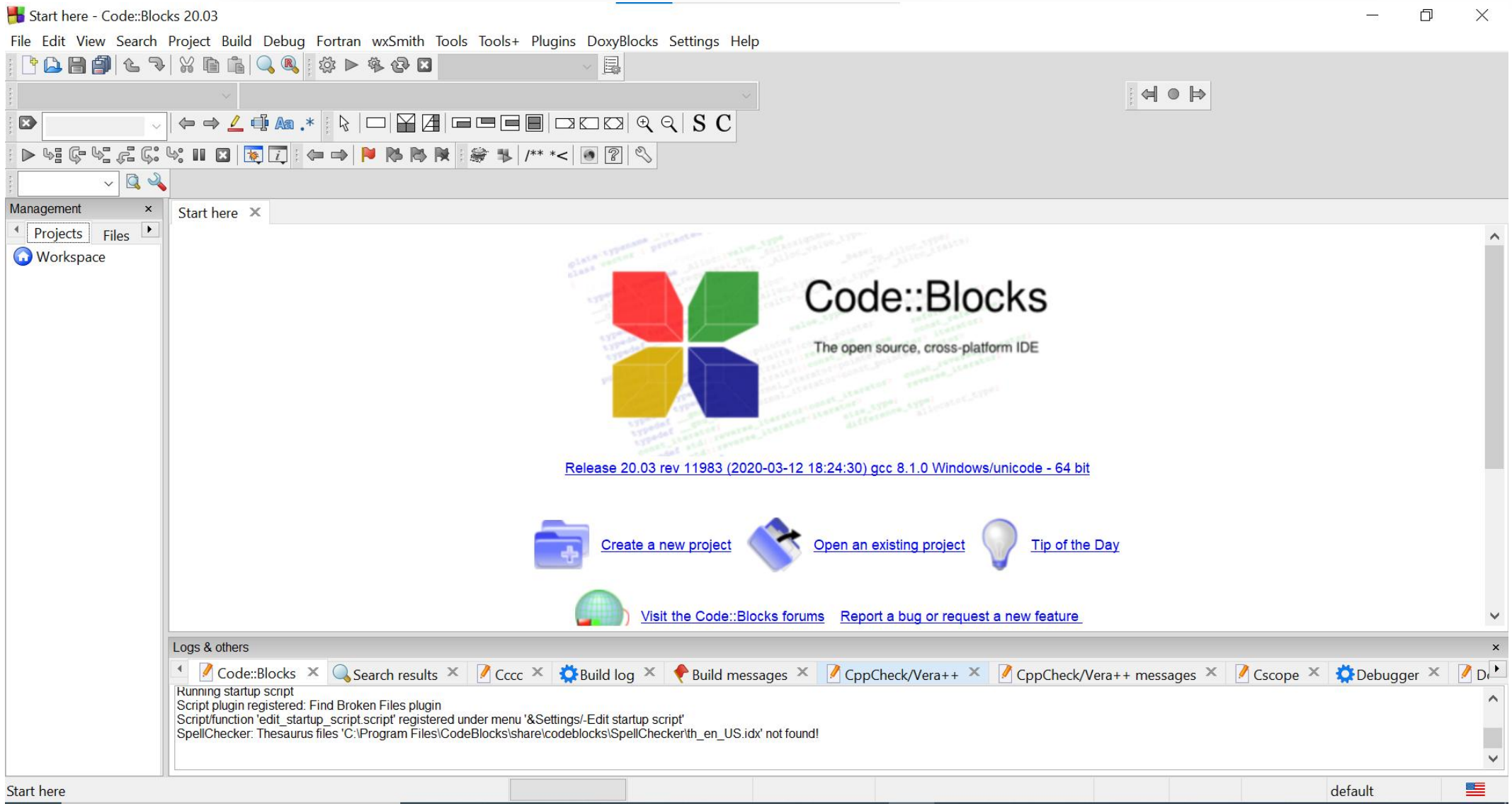
05 Pointers and Array

06 Structures and Strings

07 Input and Output

XIAMEN UNIVERSITY

Codeblocks IDE



Codeblocks Download Link:

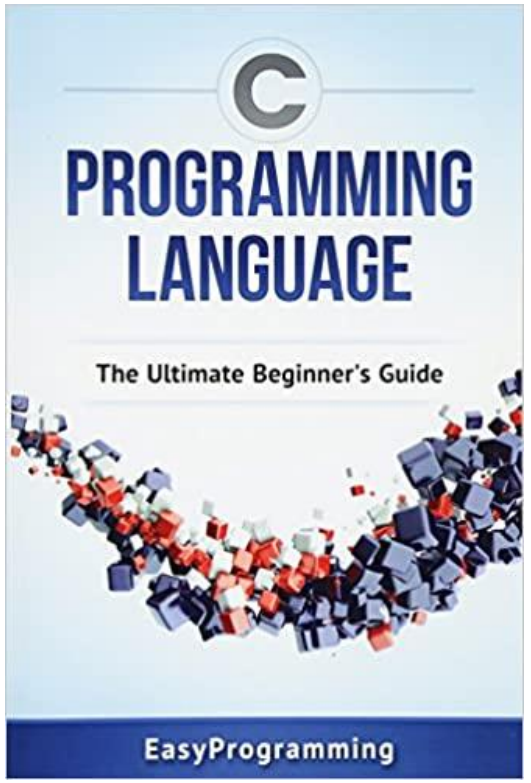
http://www.codeblocks.org/downloads/26

Attendance

- Attendance is required by Ministry of Higher Education Malaysia
- Attendance sheet should be signed on each class, including lab sessions
- Sick leave and other types leaves should go through standard procedures
- Students will be barred from sitting final exam if their attendance rate is <80% of each course).

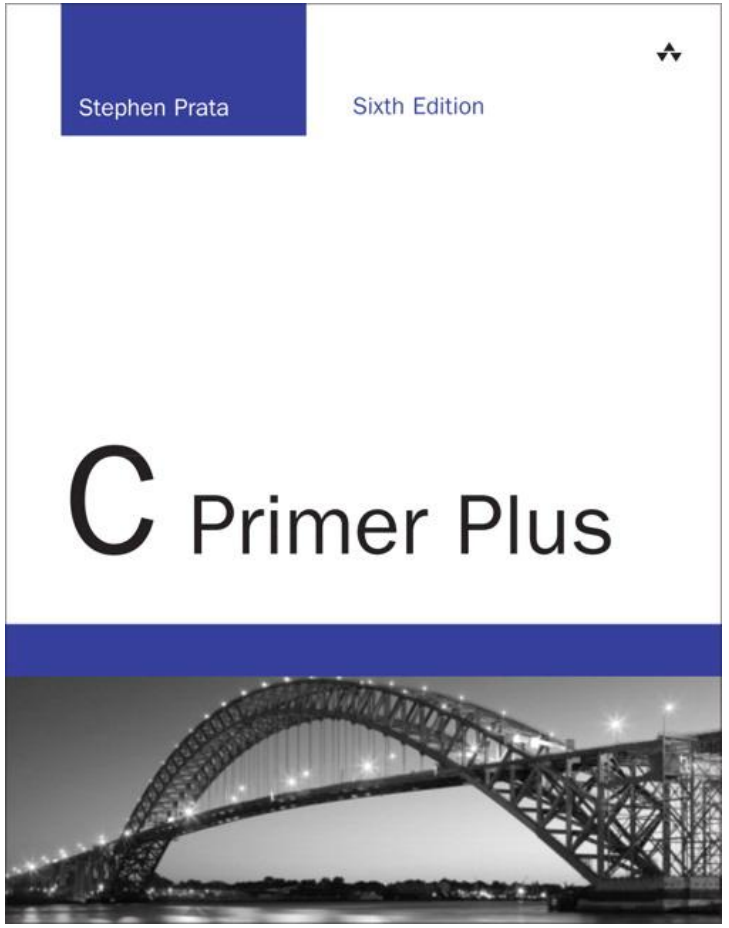
Required References

- ✓ Easy Programming, "C Programming Language: The Ultimate Beginner's Guide," CreateSpace Independent Publishing Platform (November 12, 2016).



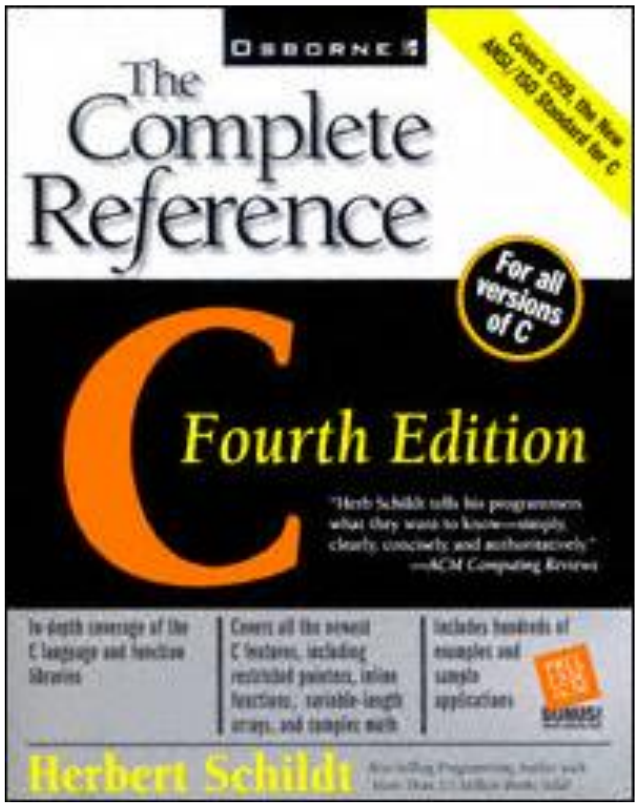
Required References

- ✓ Stephan Prata, C primer plus 6th edition. Pearson, USA. (2014)



Required References

- ✓ Herbert Schildt, C The Complete Reference, Fourth Edition, ISBN-13: 978-0072121247, April 26th 2000, The McGraw-Hill Companies.



Chapter 1

Introduction

1.1 An Overview



What is programming?

- Process of creating instructions for computers.
- Involves: Designing & writing code, testing & debugging, and maintaining software.
- Uses programming languages (e.g., Python, C++).
- Automates tasks, solves problems, and builds software.
- Requires logical thinking and problem-solving skills.



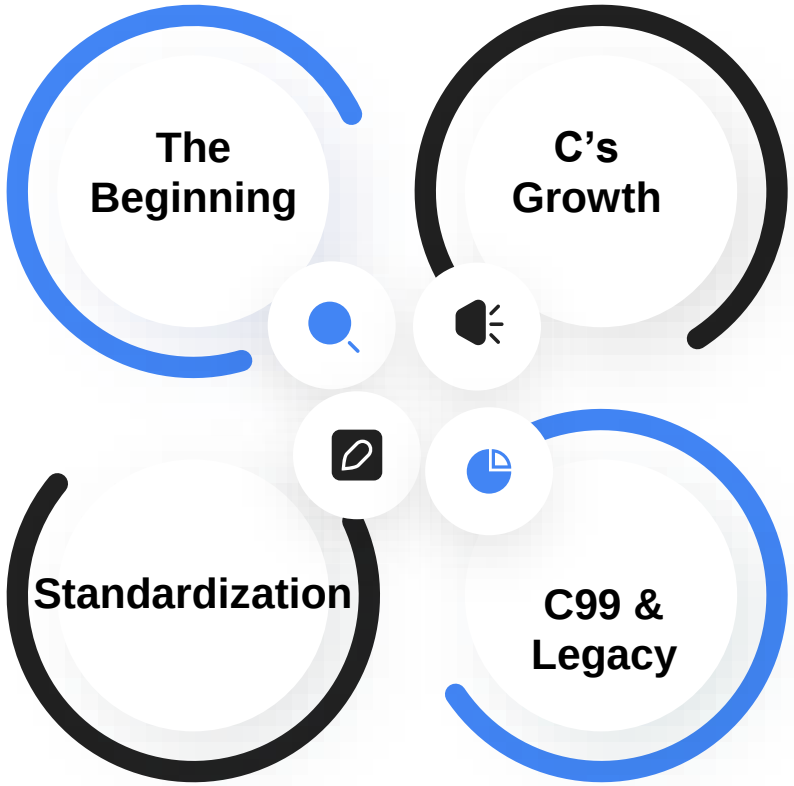
Common Programming Languages

JavaScript, Python, Go, Java, Kotlin, PHP, C#, Swift, R, Ruby, C , C#, and C++, MATLAB, SQL, HTML, CSS, NoSQL, Perl

The Story of C Programming

Dennis Ritchie developed **C** in the 1970s on the **DEC PDP-11**, inspired by older languages **BCPL** and **B.**

The language became widely used, especially with the **Unix** system. Its first official description was in *C Programming Language* by **Kernighan & Ritchie** (1978).



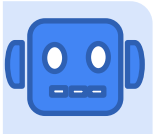
In 1983, work began to standardize C, and in **1989**, the **ANSI C** standard (C89) was officially adopted.

In 1999, **C99** introduced new features, ensuring C's continued relevance. C remains the foundation of modern programming, influencing languages like **C++** and beyond.

Application examples



Operating Systems Development
(Due to its low-level capabilities and efficiency)



Robotics
Robot Operating Systems (ROS) for building robotic systems, Real-time Control Systems, and Sensor and Actuator Interface, for example industriel automation and autonomous vehicles.
(Deterministic behavior



Compiler or interpreter & system services
Such as GCC (GNU Compiler Collection), Clang (LLVM compiler infrastructure), and Python's CPython interpreter, are written in C due to its speed and control over system resources.



Embedded Systems
(Due to its efficiency and direct hardware manipulation are crucial.)



Single Chip Modules (SCMs) and drive program
C is often the language of choice for programming these devices due to its efficiency, low-level capabilities, and ability to directly manipulate hardware. C is used to write firmware that controls the behavior of these chips



Scientific computing & simulations
Aerodynamics
C is favored for its performance and ability to provide low-level control over computational tasks.

1.3. 7 steps in C programming

The process of C programming typically follows seven key steps, ensuring that you can write, compile, and execute a C program efficiently

01 Define the Problem

02 Plan the Solution (Algorithm Design)

03 Write the Code (Implementation)

04 Compile the Code

05 Execute the Program

06 Debug the Code

07 Test and Improve

1.4 A Simple Example

Ex1.

```
#include <stdio.h>

int main() {
    // Print a simple message
    printf("Hello, World!\n");
    return 0;
}
```

Ex 2:

```
#include <stdio.h>

int main() {
    // Declare variables
    int a = 8;
    int b = 4;
    int result;

    // Perform addition
    result = a + b;

    // Display the result
    printf("The sum of %d and %d is: %d\n", a, b, result);

    return 0;
}
```

1.5 The Structure of a Simple Program

01

Pre-processor Directive

- Includes necessary libraries.
- **Example:**
- **#include <stdio.h>**

(This allows us to use input/output functions like printf).

02

Main Function:

- Every C program must have a **main()** function.
- The program starts execution from here.

03

Variable Declarations

- Variables are declared to store data.
- **Example**: **int a, b, result;**

1.5 The Structure of a Simple Program

04

Body of the Program

- The main logic, including assignments and operations.
- **Example: result = a + b;**

05

Output

- Use **printf()** to display output to the user.
- **Example**: **printf("Result: %d", result);**

06

Return Statement

- Ends the program and returns a value (typically **0**).
- **Example**: **return 0;**

1.5 The Structure of a Simple Program

This structure is typical of most C programs and gives you a clear outline for building basic programs.

```c
#include <stdio.h>    // Preprocessor directive

int main() {          // Main function
    int a = 5;        // Variable declarations
    int b = 10;
    int result = a + b;  // Operations

    printf("Sum: %d\n", result);  // Output

    return 0;          // Return statement
}
```

A simple example in code



1.6 Introducing Debugging

How are computer programs written?

- Programmers write instructions in various programming languages, some directly understandable by computers and others requiring intermediate translation steps.

- The programming languages can be divided into three types:
  1. Machine languages
  2. Assembly languages
  3. High-level languages

Machine languages

- Machine languages generally consist of strings of numbers (ultimately reduced to 1s and 0s) that instruct computers to perform their most elementary operations one at a time.
- Any computer can directly understand only its own machine language, defined by its hardware design.
- However, such languages are cumbersome for humans.

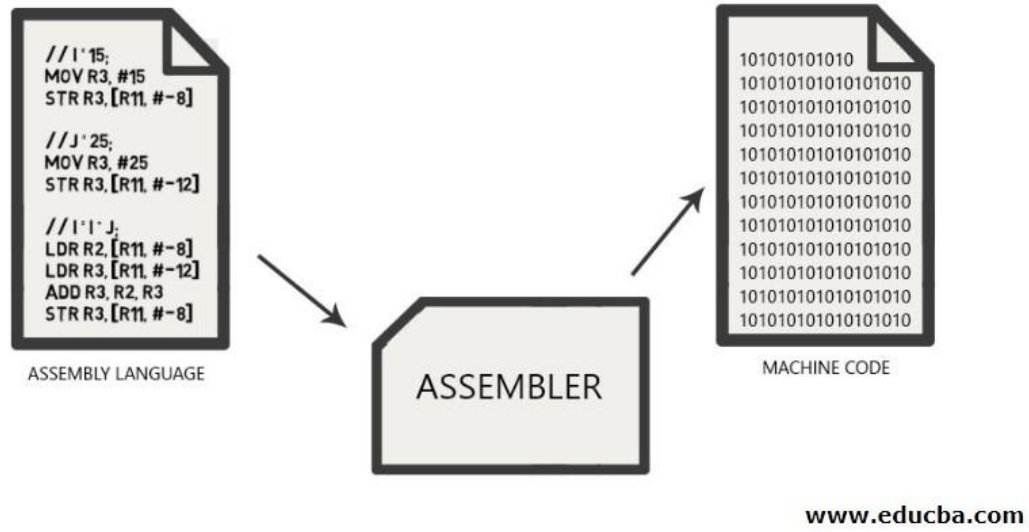
Example of machine-language

Here's what a program-fragment looks like:

```
10100001 10111100 10010011 00000100
00001000 00000011 00000101 11000000
10010011 00000100 00001000 10100011
11000000 10010100 00000100 00001000
```

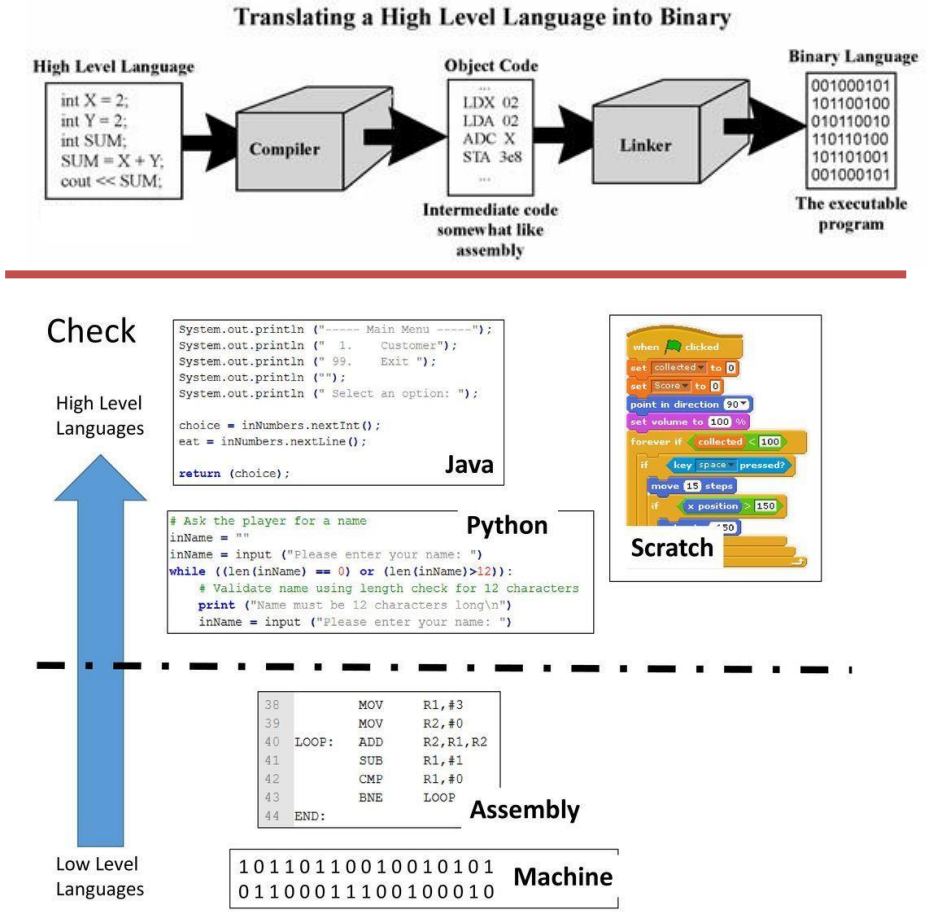
Assembly languages

- Programming in machine language was simply too slow and tedious for most programmers. Instead of using the strings of numbers that computers could directly understand, programmers began using English-like abbreviations to represent elementary operations.
- These abbreviations formed the basis of assembly languages.
- Translator programs called **assemblers** were developed to convert early assembly-language programs to machine language at computer speeds.

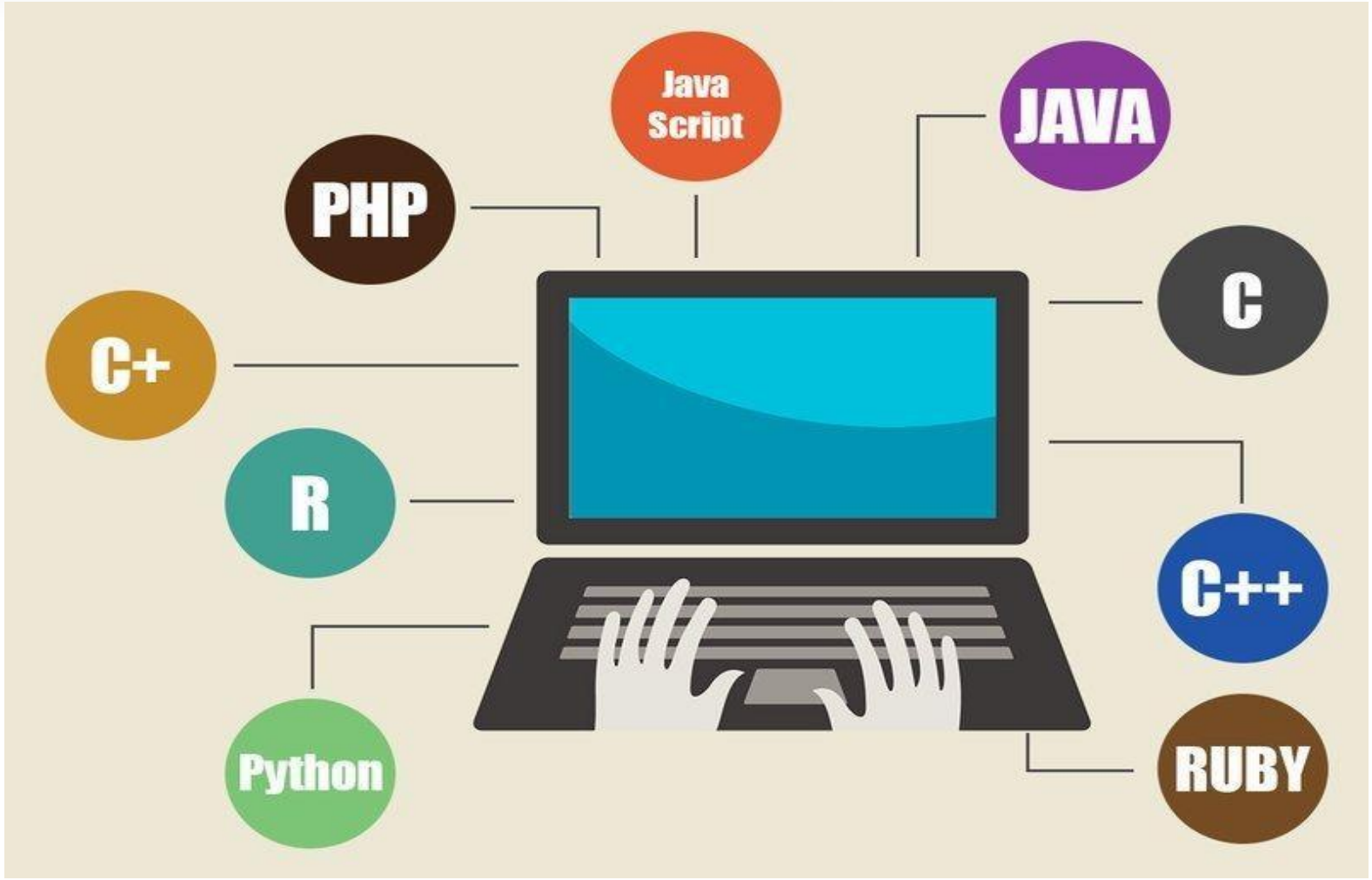


High Level Language

- With the advent of assembly languages, computer usage increased rapidly, but programmers still had to use numerous instructions to accomplish even the simplest tasks.
- To speed the programming process, high-level languages were developed in which single statements could be written to accomplish substantial tasks.
- Translator programs called **compilers** convert high-level language programs into machine language.
- High-level languages allow you to write instructions that look almost like everyday English and contain commonly used mathematical notations.



Examples of High Level Languages



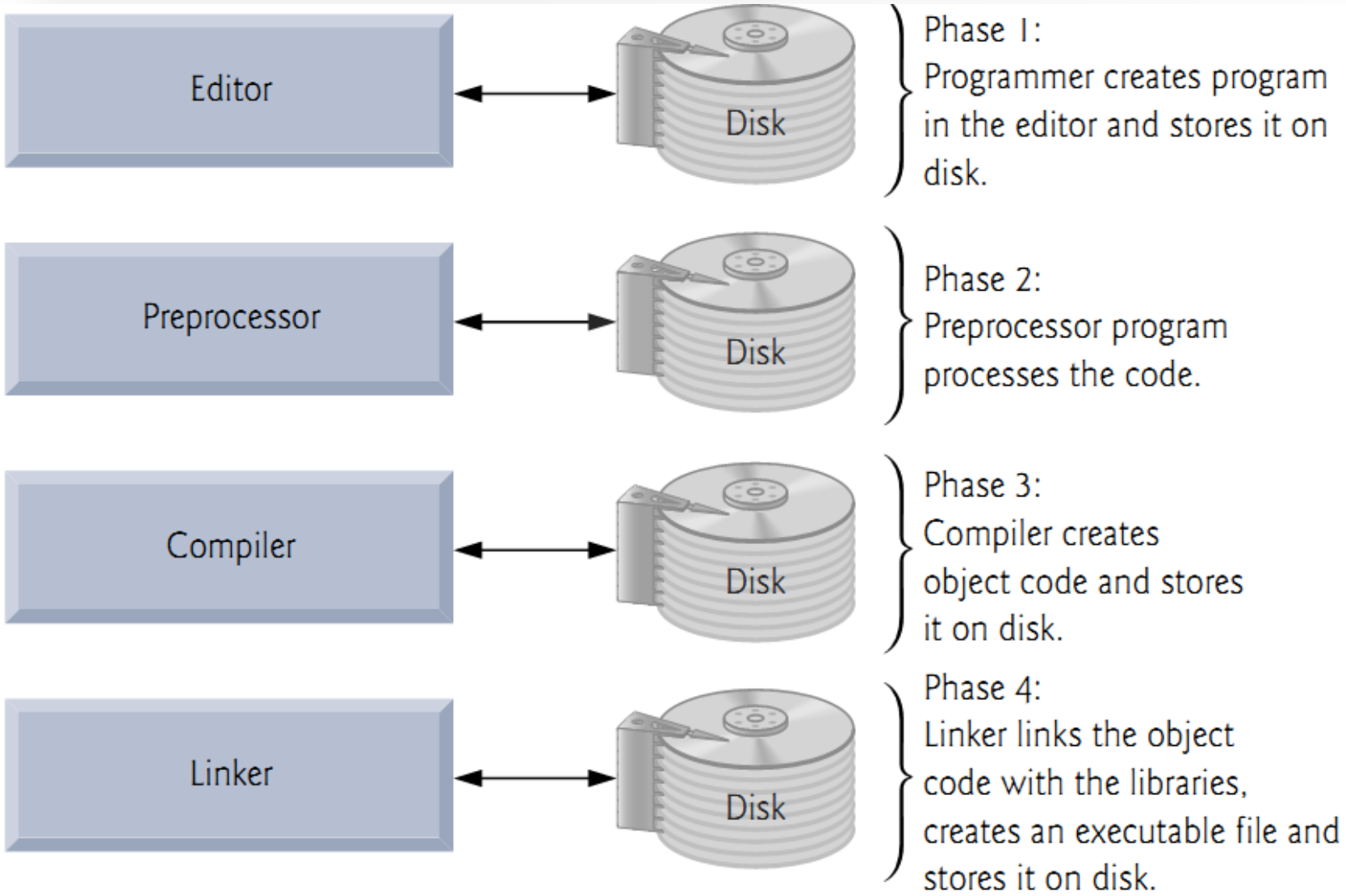
C Programming Language

- C is one of the high-level programming languages.
- C is a general-purpose programming language created by Dennis Ritchie at the Bell Laboratories in 1972.
- The C programming language turns 52 years old in 2024. Yet despite its long history, C remains one of the top "most-used" programming languages

Sep 2024	Sep 2023	Change	Programming Language	Ratings	Change
1	1		Python	20.17%	+6.01%
2	3	↑	C++	10.75%	+0.09%
3	4	↑	Java	9.45%	-0.04%
4	2	↓	C	8.89%	-2.38%
5	5		C#	6.08%	-1.22%
6	6		JavaScript	3.92%	+0.62%
7	7		Visual Basic	2.70%	+0.48%
8	12	⇈	Go	2.35%	+1.16%
9	10	↑	SQL	1.94%	+0.50%
10	11	↑	Fortran	1.78%	+0.49%

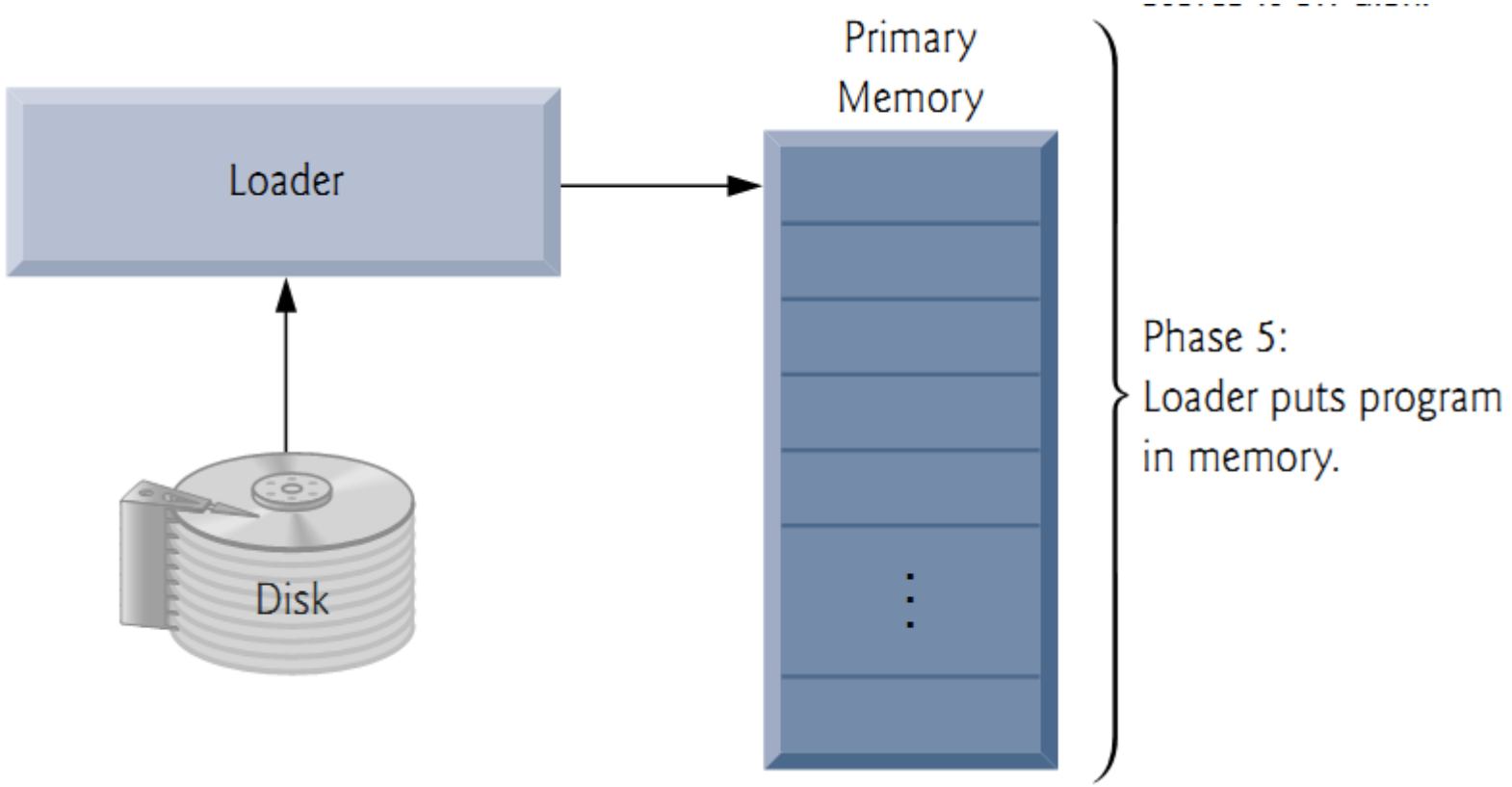
TIOBE Index (2024)

C Program-Development Environment



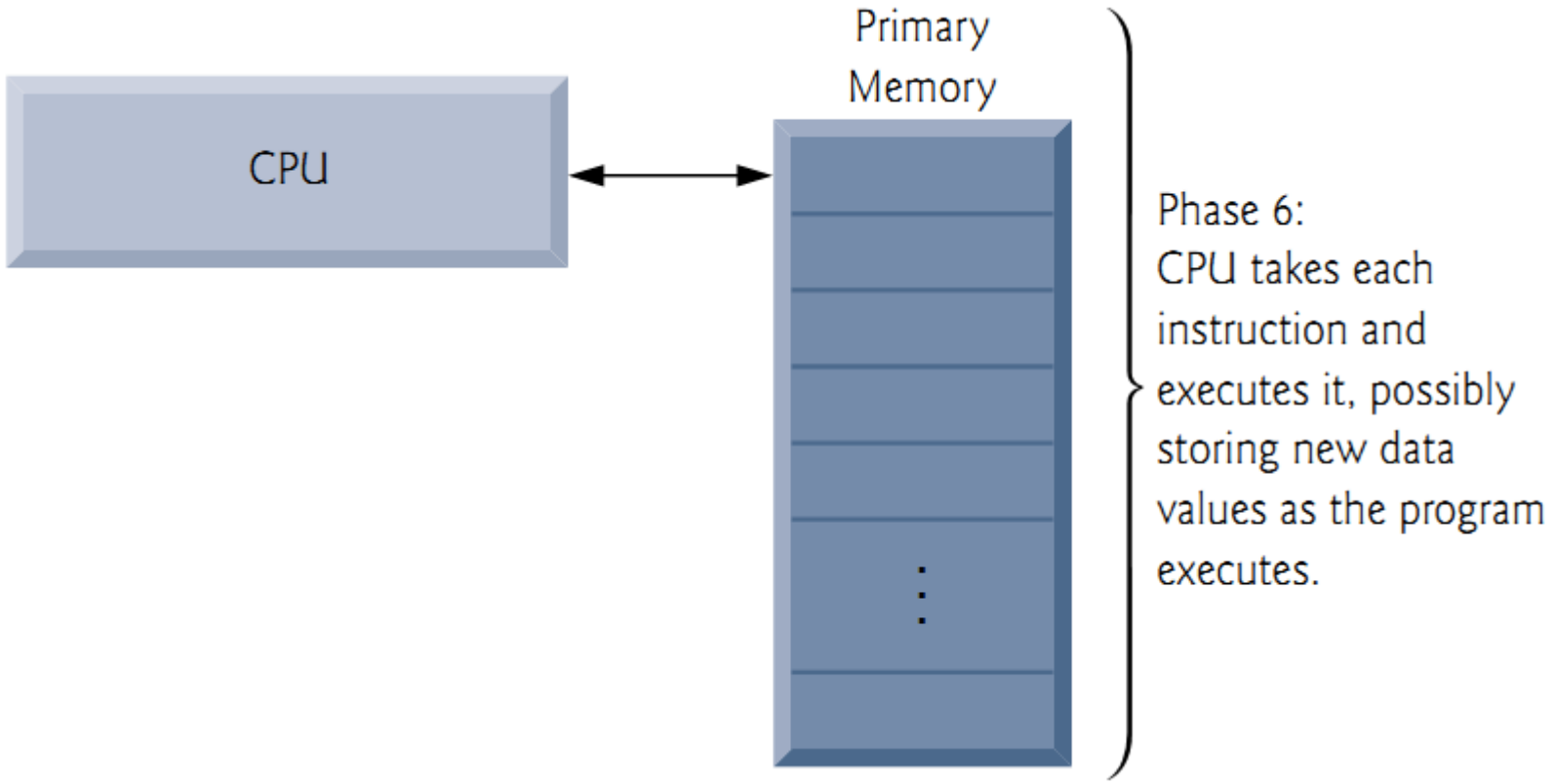
Source: p.47, Deitel & Deitel *C How to Program*, 8th edition

Debugging



Source: p.47, Deitel & Deitel *C How to Program*, 8th edition

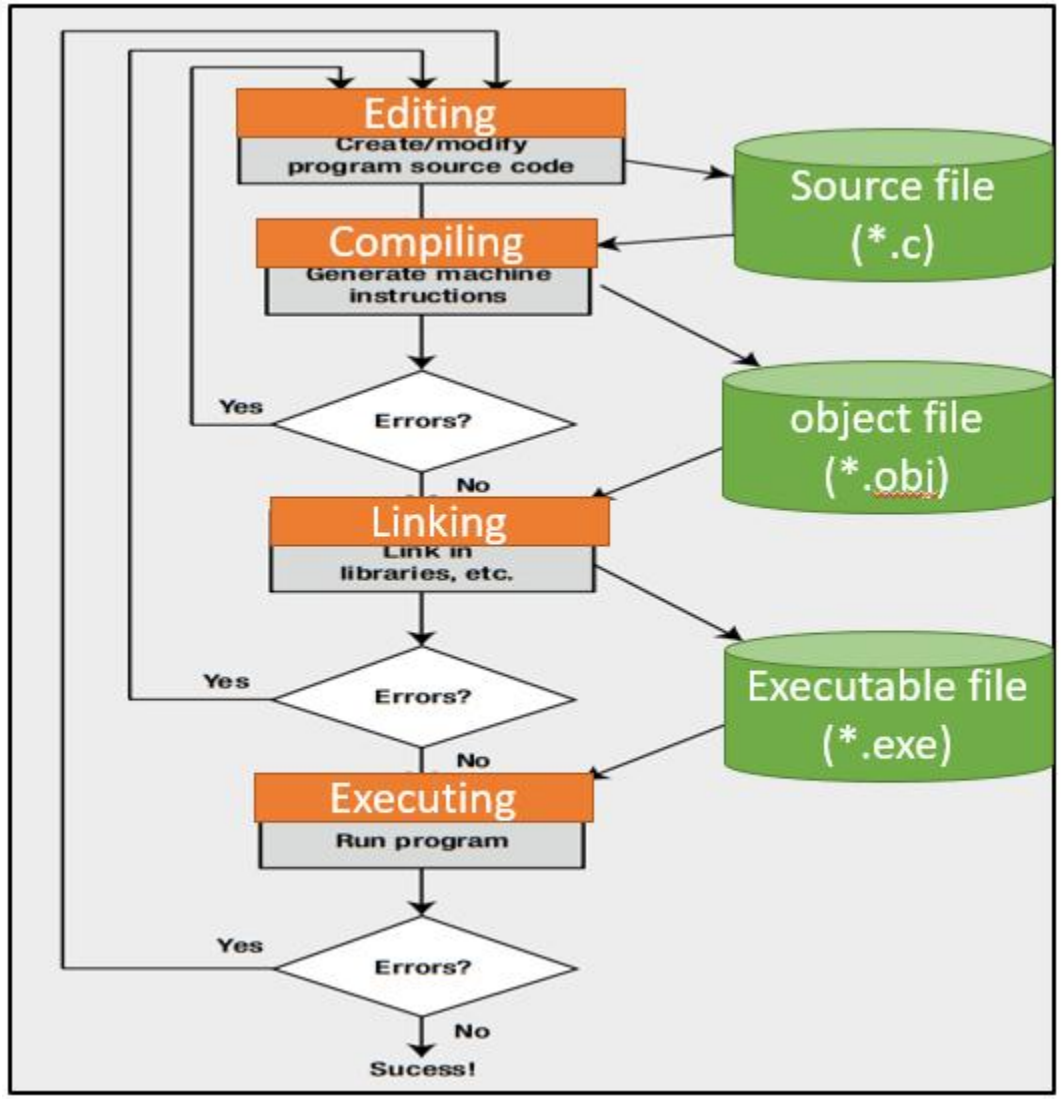
Execution of Instructions



Source: p.47, Deitel & Deitel *C How to Program*, 8th edition

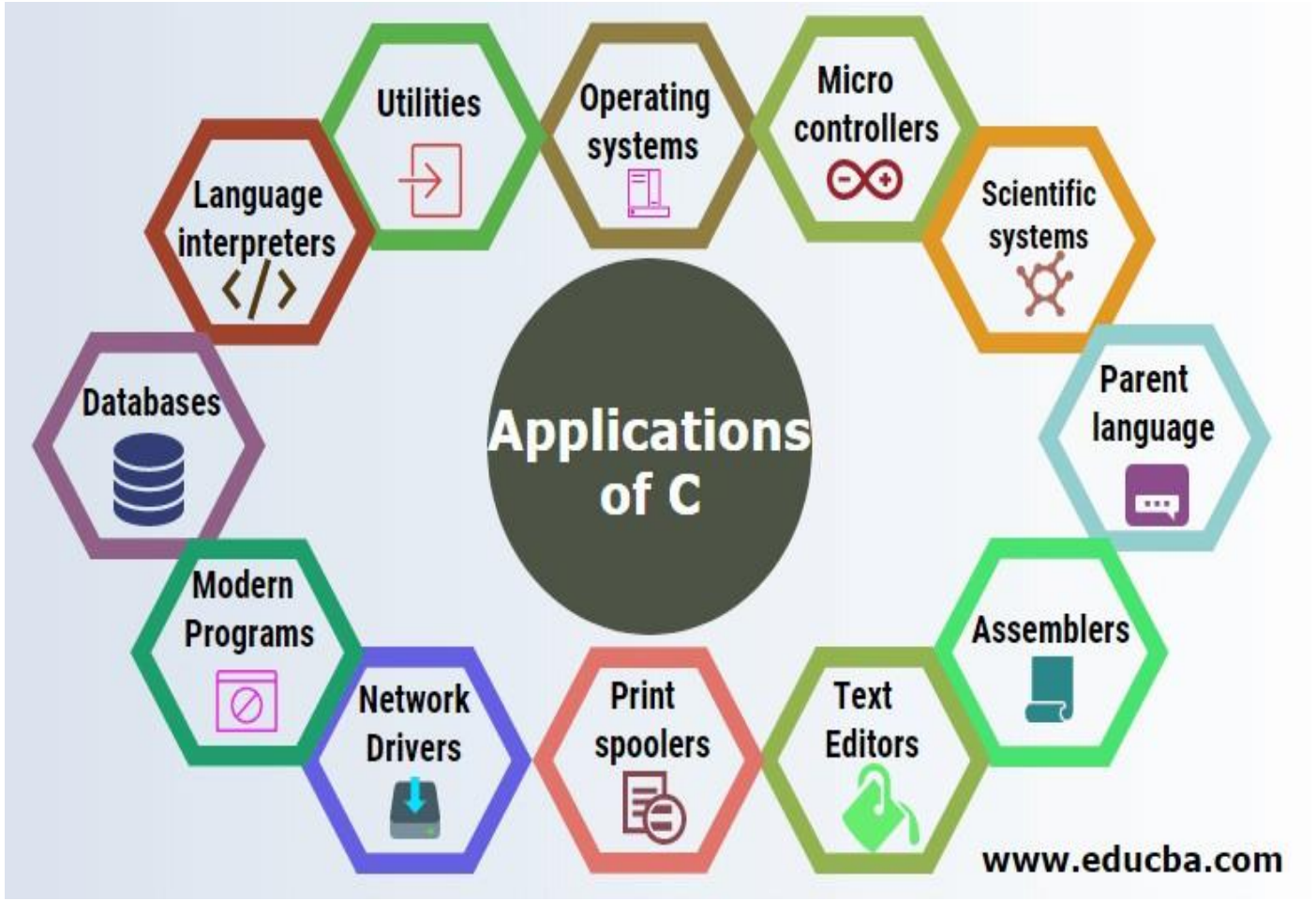
The overall C Program Development Process

- Core Flowchart



Areas of Application of C Language

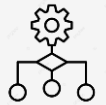
- C is a high-level general-purpose programming language that can be applied to many different classes of problems.
- It can be used for processing text, numbers, scientific data, or anything else which one might save on a computer.



Features of C Programming Language

- **It is simple to understand**
  - C programming language uses blocks to separate pieces of code performing different tasks.
  - This helps make programming easier and keeps the code clean.
- **It is used in embedded programming**
  - C is used in embedded programming, which is used to control micro-controllers. These micro- controllers are used in robotics, computers, and automation.
- **It is fast**
  - The programs that you write in C compile and execute much faster than those written in other languages such as COBOL.
- **It forms the basis for C++ and Java**
  - Java and C++ are amongst the most widely used programming languages today. They derive their syntax, and concepts from the C programming language.

Features of C Programming Language

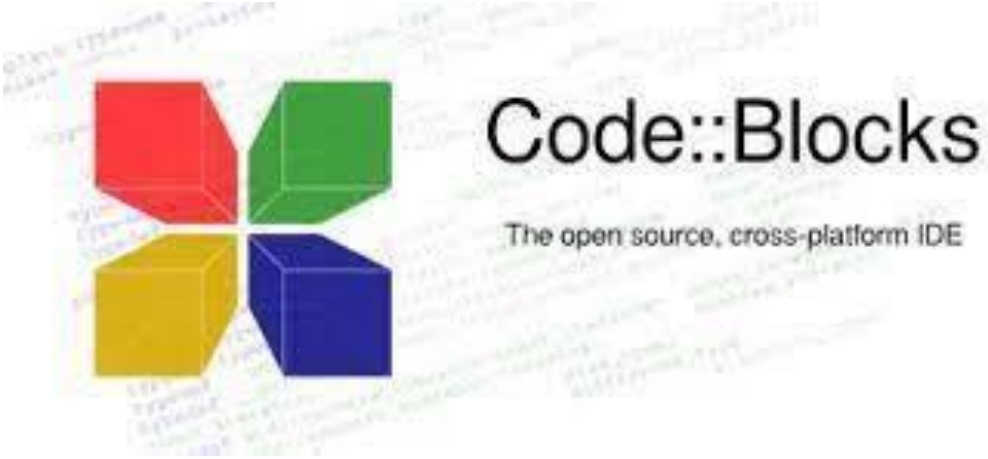


- **It is extensible**
  - Customized or user-defined codes can be added to the preexisting codes.
- **It has function-rich libraries**
  - Provides ready-to-use tool, in the form of built-in functions.
- **It is modular**
  - It is a structured language with modularity. Ease for development and maintenance.

Installing Compiler (Windows)

https://jmeubank.github.io/tdm-gcc/

https://www.codeblocks.org/downloads/



We are going to use Code::Blocks as our default programming editor.

Other alternative IDEs, including free online IDEs, can be used.



Thank you

A & Q